Buckeye Marketplace

AMIS 4630 — Spring 2026

Milestone 6: Production Deployment

Author: Ilyas Jama

Repository: github.com/1lyasjama/amis4630-spring26-jama

Frontend: https://buckeye-frontend-jama.azurewebsites.net

API: https://buckeye-api-jama.azurewebsites.net

README — Project Overview, Tech Stack, Setup, Deployment

Source: `README.md`

A full-stack marketplace web app for Ohio State students to buy and sell items safely. Built across six milestones for AMIS 4630, Spring 2026.

> **Live demo (Milestone 6):**
> - Frontend: `https://buckeye-frontend-jama.azurestaticapps.net`
> - API: `https://buckeye-api-jama.azurewebsites.net`
> - Swagger: `https://buckeye-api-jama.azurewebsites.net/swagger`

Project description

Buckeye Marketplace lets students browse listings, add items to a cart, register and log in, place orders, and view their order history. Admins can manage the catalog and update order statuses. The system runs entirely on Azure (Static Web Apps + App Service + Azure SQL Database) with continuous deployment from GitHub Actions.

Features

- **Product catalog** — browse and view product detail pages.
- **Shopping cart** — add, update quantity, remove items, persisted to the database per user.
- **Authentication** — register, login, logout, JWT bearer tokens with silent refresh-token rotation.
- **Authorization** — `User` and `Admin` roles enforced both in the client (route guards) and on the server (`[Authorize(Roles = "...")]`).
- **Order placement** — checkout with shipping address, generates a confirmation number, clears the cart, snapshots line items.
- **Order history** — users see their own orders, scoped by JWT.
- **Admin dashboard** — product CRUD and order-status management.
- **Production deployment on Azure** with HTTPS, SPA fallback routing, and configurable CORS origins.
- **CI/CD on GitHub Actions** — every push runs the full test suite; green builds auto-deploy to Azure.

Technology stack

Layer	Technology	Version
Frontend	React + TypeScript	React 19, TS 5.9
Build tool	Vite	7.x
Routing	React Router	7.x
Frontend tests	Vitest + React Testing Library	Vitest 4.x
E2E	Playwright	1.59
Backend	ASP.NET Core Web API	.NET 10
ORM	Entity Framework Core	10.x
Auth	ASP.NET Core Identity + JWT bearer	10.x
Validation	FluentValidation	11.x
Backend tests	xUnit + WebApplicationFactory	xUnit 2.x
Database (dev)	SQLite	provider 10.x
Database (prod)	Azure SQL Database (Basic tier)	—
Hosting (frontend)	Azure Static Web Apps	—
Hosting (API)	Azure App Service (Linux F1)	—
CI/CD	GitHub Actions	—

Local development

Prerequisites

- .NET 10 SDK
- Node.js 20+ and npm

1. Set the JWT signing key (one-time, per machine)

```
cd api
dotnet user-secrets set "Jwt:Key" "ThisIsADevelopmentOnlySecretKey_ChangeInProduction_MinimumLength32Characte
```

2. Run the API

```
cd api
dotnet run
```

The API listens on **http://localhost:5023**. Swagger is at `http://localhost:5023/swagger`.

The local database is SQLite (`api/BuckeyeMarketplace.db`). The schema is created automatically on first run, and `SeedData.cs` seeds:

- Two roles (`Admin`, `User`)
- A test admin: `admin@buckeye.test` / `Admin123!`
- A test user: `user@buckeye.test` / `User1234!`
- 10 sample product listings

3. Run the frontend

```
cd frontend
npm install
npm run dev
```

The app starts on **http://localhost:5173**. The frontend reads `VITE_API_URL` at build time; in dev it defaults to the local API.

Production deployment

See docs/deployment.md for the full runbook. The short version:

- Frontend → **Azure Static Web Apps**, deployed by `.github/workflows/deploy-frontend.yml`.
- API → **Azure App Service** (Linux, .NET 10), deployed by `.github/workflows/deploy-api.yml`.
- Database → **Azure SQL Database** (Basic tier).
- HTTPS is enforced on both services.
- Secrets:
  - `Jwt:Key` lives in App Service application settings.
  - SQL connection string lives in App Service connection strings.
  - The Azure publish profile and SWA token live in GitHub repo secrets.

CI/CD

Every push and pull request runs `.github/workflows/ci.yml`:

- `dotnet test` (17 tests across unit + integration + auth)
- `npm test -- --run` (13 frontend unit/component tests)
- `npm run build` (production frontend bundle)

Pushes to `main` that touch the API trigger `deploy-api.yml`; pushes that touch the frontend trigger `deploy-frontend.yml`. Tests gate every deploy.

Environment variables

API (App Service application settings in production; user-secrets locally)

Setting	Required	Notes
`Jwt:Key`	yes	32+ char signing key. App throws on startup if missing.
`Jwt:Issuer`	yes	Production hostname.
`Jwt:Audience`	yes	Production hostname.
`Cors:AllowedOrigins:0..N`	prod	Frontend origins to allow (in addition to localhost).
`ConnectionStrings:Default`	dev	SQLite path; defaults to `Data Source=BuckeyeMarketplace.db`.

Setting	Required	Notes
`ConnectionStrings:DefaultConnection` (App Service)	prod	Azure SQL connection string.
`AdminSeed:Email` / `AdminSeed:Password`	optional	Override the seeded admin credentials.

Frontend (Vite build-time)

Variable	Required	Notes
`VITE_API_URL`	prod	Base URL of the deployed API (no trailing `/api`). Set in `.env.production` or as a GitHub variable for the deploy workflow.

API documentation

Swagger UI ships with the API in every environment:

- Local: `http://localhost:5023/swagger`
- Production: `https://buckeye-api-jama.azurewebsites.net/swagger`

Endpoints summary

Auth (`/api/auth`)

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	none	Register; returns access + refresh tokens.
POST	`/api/auth/login`	none	Login; returns access + refresh tokens.
POST	`/api/auth/refresh`	none	Rotate a refresh token (revokes the old one).

Products (`/api/products`)

Method	Endpoint	Auth	Description
GET	`/api/products`	none	List all products.
GET	`/api/products/{id}`	none	Get product by id.
POST	`/api/products`	Admin	Create product.
PUT	`/api/products/{id}`	Admin	Update product.
DELETE	`/api/products/{id}`	Admin	Delete product.

Cart (`/api/cart`)

Method	Endpoint	Auth	Description
GET	`/api/cart`	User	Get current user's cart.
POST	`/api/cart`	User	Add item.
PUT	`/api/cart/{cartItemId}`	User	Update item quantity.
DELETE	`/api/cart/{cartItemId}`	User	Remove a single item.
DELETE	`/api/cart/clear`	User	Clear all items.

Orders (`/api/orders`)

Method	Endpoint	Auth	Description
POST	`/api/orders`	User	Create order from cart; clears the cart.
GET	`/api/orders/mine`	User	Current user's orders (JWT-scoped).
GET	`/api/orders/{id}`	User	Single order; only if owned by the JWT user, or admin.
GET	`/api/orders`	Admin	All orders.

Method	Endpoint	Auth	Description
PUT	`/api/orders/{orderId}/status`	Admin	Update order status.

Project structure

```
amis4630-spring26-jama/
├── .github/workflows/             # CI + deploy pipelines
│   ├── ci.yml
│   ├── deploy-api.yml
│   └── deploy-frontend.yml
├── api/                           # ASP.NET Core 10 Web API
│   ├── Controllers/               # Auth, Products, Cart, Orders
│   ├── Data/                      # AppDbContext + seed data
│   ├── Dtos/                      # Request / response shapes
│   ├── Migrations/                # EF Core migrations
│   ├── Models/                    # Domain entities
│   ├── Services/                  # JwtTokenService, OrderCalculator, PasswordRules
│   ├── Validators/                # FluentValidation rules
│   └── Program.cs                 # Composition root, security middleware
├── BuckeyeMarketplaceApi.Tests/   # xUnit unit + integration tests
├── frontend/                      # React + TypeScript SPA
│   ├── e2e/                       # Playwright spec
│   ├── src/
│   │   ├── components/            # Header, ProductCard, LoginForm, ...
│   │   ├── context/               # AuthContext, CartContext
│   │   ├── pages/                 # Catalog, Detail, Cart, Checkout, Admin*
│   │   ├── services/              # apiFetch, authService, cartService, orderService
│   │   ├── test/                  # Vitest unit/component tests
│   │   ├── types/                 # TypeScript interfaces
│   │   └── utils/                 # validation helpers
│   ├── staticwebapp.config.json   # SPA fallback + security headers (SWA)
│   ├── playwright.config.ts
│   └── vite.config.ts
├── docs/
│   ├── adr/                       # Architecture Decision Records
│   ├── system-architecture.md     # High-level architecture (M6)
│   ├── database-erd.md            # Database schema (M6)
│   ├── deployment.md              # Production deployment runbook
│   ├── test-plan.md               # E2E test plan (M6)
│   ├── qa-report.md               # QA results for the M6 build
│   ├── user-guide.md              # Shopper user guide
│   ├── admin-guide.md             # Admin guide
│   ├── ai-reflection.md           # M6 AI tool reflection
│   ├── ai-usage-m4.md             # M4 AI usage notes
│   ├── e2e-run.md                 # M5 Playwright agent run notes
│   └── screenshots/
├── README.md                      # this file
├── SUBMISSION.md                  # Submission notes (M5 + M6)
├── AI-USAGE.md                    # AI usage summary
├── CHANGELOG.md                   # Per-milestone changelog
└── amis4630-spring26-jama.sln
```

Documentation index

Doc	Purpose
docs/system-architecture.md	High-level architecture, request flows, deployment topology
docs/database-erd.md	ERD + entity descriptions
docs/deployment.md	Azure provisioning + CI/CD runbook
docs/test-plan.md	What we test, how, and where
docs/qa-report.md	Recorded results from the M6 build
docs/user-guide.md	Shopper-facing user guide

Doc	Purpose
docs/admin-guide.md	Admin guide for product / order management
docs/ai-reflection.md	Cross-milestone reflection on AI use
docs/adr/	Architecture Decision Records
SUBMISSION.md	Submission notes + test credentials
AI-USAGE.md	AI usage summary across milestones
CHANGELOG.md	Per-milestone change log

Testing

```
# Backend (unit + integration via WebApplicationFactory + in-memory provider)
dotnet test amis4630-spring26-jama.sln --configuration Release

# Frontend unit / component
cd frontend && npm test -- --run

# End-to-end happy path (Playwright)
cd frontend && npx playwright install chromium && npx playwright test
```

`dotnet build` produces zero warnings; all 17 backend tests, 13 frontend tests, and the 1 Playwright E2E spec pass on a fresh clone.

Architecture Decisions (ADRs)

Stored in docs/adr/:

- ADR-001 Frontend: React + TypeScript
- ADR-002 Backend: .NET Web API
- ADR-003 Database: Azure SQL (relational)
- ADR-004 Authentication: OSU/Microsoft Login (note: implementation shipped as ASP.NET Core Identity + JWT; OSU SSO would be a future swap-in)
- ADR-005 Cloud: Azure deployment

AI usage

A cross-milestone reflection lives at docs/ai-reflection.md. Per-milestone notes:

- **Milestone 3** — see the table in AI-USAGE.md and earlier git history.
- **Milestone 4** — docs/ai-usage-m4.md.
- **Milestone 5** — AI-USAGE.md; Playwright agent run notes in docs/e2e-run.md.
- **Milestone 6** — docs/ai-reflection.md.

All AI-assisted code was reviewed, edited, and tested before being committed. The engineering judgment — auth boundaries, secrets handling, test strategy, deployment topology — is mine.

System Architecture

Source: `docs/system-architecture.md`

Overview

Buckeye Marketplace is a full-stack marketplace web app for Ohio State students. The architecture is a classic three-tier SPA + REST API deployment, hosted on Azure, with a clear split between presentation, application, and data layers.

This document supersedes the Milestone 1 sketch and reflects the system as it actually shipped through Milestones 3–6.

High-Level Architecture (production)

```
flowchart LR

  subgraph Browser
    U[Student / Admin]
  end

  subgraph Edge
    SWA[Azure Static Web Apps<br/>React + TypeScript SPA]
  end

  subgraph App
    API[Azure App Service<br/>ASP.NET Core 10 Web API]
  end

  subgraph Identity
    JWT[JWT Bearer Tokens<br/>HS256, 8h access / 14d refresh]
  end

  subgraph Data
    DB[(Azure SQL Database<br/>BuckeyeMarketplaceDb)]
  end

  subgraph DevOps
    GH[GitHub Actions CI/CD]
  end

  U -->|HTTPS| SWA
  SWA -->|REST + Bearer JWT| API
  API -->|EF Core / TDS| DB
  API -.->|issues / validates| JWT
  GH -->|deploy on push to main| SWA
  GH -->|deploy on push to main| API
```

Architectural Layers

1. Presentation Layer (frontend)

- **React 19 + TypeScript**, built with Vite, hosted on Azure Static Web Apps.
- Client-side routing via React Router 7. `staticwebapp.config.json` rewrites unknown paths to `/index.html` so deep links survive refresh.
- Auth state lives in a `useReducer`-based `AuthContext`; the JWT and refresh token are stored in `localStorage`.
- Every API call goes through a single `apiFetch` helper that injects the bearer token, transparently retries once after a silent refresh on 401, and surfaces typed `ApiError`s to UI components.
- Protected routes (`/cart`, `/checkout`, `/orders`, `/admin/*`) are gated client-side with a `<ProtectedRoute>` component, but every protection is also enforced server-side.

2. Application Layer (backend)

- **ASP.NET Core 10 Web API**, hosted on a Linux Azure App Service.
- Controllers per resource: `AuthController`, `ProductsController`, `CartController`, `OrdersController`. Business logic that doesn't belong in a controller (`OrderCalculator`, `JwtTokenService`, `PasswordRules`) lives in `Services/` so

it can be unit-tested in isolation.

- **Authentication** via ASP.NET Core Identity + `JwtBearer`. The signing key is read from `Jwt:Key` (App Service application setting in production, `dotnet user-secrets` locally). Startup throws if the key is missing.
- **Authorization** is two-tier: `[Authorize]` for any logged-in user, `[Authorize(Roles = "Admin")]` for admin endpoints. User-scoped queries always read the user id from `ClaimTypes.NameIdentifier` on the JWT — never from the URL or body — to defend against broken-object-level authorization (OWASP API1).
- **Validation** uses FluentValidation; request DTOs have explicit validators (`AddToCartRequestValidator`, `UpdateCartItemRequestValidator`, etc.).
- **Security headers** (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) are added by middleware. HTTPS redirect is enabled in non-Development environments.
- **CORS** is configured in `Program.cs`; the production frontend origin is added via the `Cors:AllowedOrigins` configuration array, not by code changes.

3. Data Layer

- **Azure SQL Database** in production; **SQLite** for local development (so contributors don't need an Azure connection to run the app).
- Access goes through Entity Framework Core 10 LINQ — no `FromSqlRaw`, no string-interpolated SQL anywhere.
- Schema is migration-driven (`api/Migrations/`); the API runs `db.Database.MigrateAsync()` on startup so production stays in sync.
- Identity tables (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, etc.) live in the same database alongside the domain tables; this is a conscious tradeoff to keep things simple for a class project.

4. DevOps Layer

- **CI/CD**: three GitHub Actions workflows in `.github/workflows/`.
  - `ci.yml` — build + test on every push and PR (gate for `main`).
  - `deploy-api.yml` — builds, tests, publishes, and deploys the .NET API to App Service via the Azure publish profile.
  - `deploy-frontend.yml` — runs Vitest, builds the production bundle, and deploys to Azure Static Web Apps via the SWA token.
- **Tests gate every deploy.** Both deploy workflows refuse to ship if `dotnet test` or `npm test -- --run` fails.

Key Request Flows

Login → protected request

1. Browser → `POST /api/auth/login` with email + password.
2. API verifies via `UserManager.CheckPasswordAsync`; on success issues a signed JWT and a refresh token, persists the refresh token row.
3. Frontend stores both tokens in `localStorage`.
4. All subsequent requests carry `Authorization: Bearer <jwt>`. On `401`, `apiFetch` calls `POST /api/auth/refresh` once, gets a new pair, and replays the original request.

Place an order

1. `POST /api/orders` with the shipping address (auth required).
2. Controller reads the user id from the JWT, loads that user's cart, computes line totals via `OrderCalculator`, persists `Order` + `OrderItem`s in a single transaction, and clears the cart.
3. Response includes the generated confirmation number.

Admin order management

1. `GET /api/orders` (Admin role required) returns every order.
2. `PUT /api/orders/{id}/status` (Admin) updates `Order.Status`.
3. The admin SPA hits the same endpoints with the same JWT — there is no separate admin app.

Deployment topology (production)

Component	Service	Hostname (example)
SPA	Azure Static Web Apps	`https://buckeye-frontend-jama.azurestaticapps.net`

Component	Service	Hostname (example)
API	Azure App Service (Linux)	`https://buckeye-api-jama.azurewebsites.net`
Database	Azure SQL Database	`buckeye-sql-jama.database.windows.net`
CI/CD	GitHub Actions	`.github/workflows/`

See deployment.md for the full runbook.

Why this architecture

- **SPA + REST keeps the contract simple.** The frontend is a static bundle; cache headers and CDN-style hosting are essentially free on SWA.
- **JWT bearer auth scales horizontally.** No sticky sessions; App Service can scale out without a session store.
- **EF Core LINQ everywhere** removes whole classes of SQL injection bugs and lets the same model code run against SQLite locally and Azure SQL in production.
- **Tests gate deploys** so a broken `main` never ships.
- **Secrets stay out of source.** Runtime secrets live in App Service configuration; build-time secrets live in GitHub repository secrets; the application code only reads from `IConfiguration`.

Database Schema (ERD)

Source: `docs/database-erd.md`

This document reflects the database schema as it actually shipped. The Milestone 1 ERD imagined messaging, reviews, and reports; the scope landed on a marketplace **catalog + cart + order + auth** flow, which is what's modeled here.

The schema is migration-driven. Source of truth: `api/Migrations/`. EF Core runs `Database.MigrateAsync()` on startup.

Entities (shipped)

Entity	File	Notes
`ApplicationUser`	`api/Models/ApplicationUser.cs`	Subclass of `IdentityUser`; backs `AspNetUsers`.
`IdentityRole`	(framework)	Two seeded roles: `Admin`, `User`.
`Product`	`api/Models/Product.cs`	Marketplace listing.
`Cart`	`api/Models/Cart.cs`	One per user; `UserId` ties to `AspNetUsers`.
`CartItem`	`api/Models/CartItem.cs`	Line item in a cart, references `Product`.
`Order`	`api/Models/Order.cs`	Created from a cart at checkout; carries `OrderStatus` enum.
`OrderItem`	`api/Models/OrderItem.cs`	Snapshot of cart contents at order time.
`RefreshToken`	`api/Models/RefreshToken.cs`	Persists refresh tokens for rotation + revocation.

Entity-Relationship Diagram

```
erDiagram
    AspNetUsers ||--o| Cart : "owns one"
    AspNetUsers ||--o{ Order : "places"
    AspNetUsers ||--o{ RefreshToken : "issued"
    AspNetUsers }o--o{ AspNetRoles : "via AspNetUserRoles"

    Cart ||--o{ CartItem : "contains"
    CartItem }o--|| Product : "references"

    Order ||--o{ OrderItem : "contains"
    OrderItem }o--|| Product : "snapshot of"

    AspNetUsers {
        string Id PK
        string Email
        string PasswordHash
        datetime CreatedAt
    }

    Product {
        int Id PK
        string Title
        string Description
        decimal Price
        string Category
        string SellerName
        datetime PostedDate
        string ImageUrl
    }

    Cart {
        int Id PK
        string UserId FK
        datetime CreatedAt
        datetime UpdatedAt
    }

    CartItem {
```

```
        int Id PK
        int CartId FK
        int ProductId FK
        int Quantity
    }

    Order {
        int Id PK
        string UserId FK
        string ConfirmationNumber
        datetime OrderDate
        int Status
        decimal Total
        string ShippingAddress
    }

    OrderItem {
        int Id PK
        int OrderId FK
        int ProductId FK
        string ProductName
        decimal UnitPrice
        int Quantity
        decimal LineTotal
    }

    RefreshToken {
        int Id PK
        string Token UK
        string UserId FK
        datetime ExpiresAt
        datetime CreatedAt
        datetime RevokedAt
        string ReplacedByToken
    }
```

Notes on the design

- **`OrderItem` snapshots `ProductName`, `UnitPrice`, and `LineTotal`** at the moment the order is placed, so editing or deleting a `Product` later does not corrupt historical orders.
- **`RefreshToken.Token` is unique-indexed** so token reuse can be detected; revoking sets `RevokedAt` (and `ReplacedByToken` on rotation) rather than deleting the row, so we keep an audit trail.
- **Identity tables** (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserTokens`, `AspNetRoleClaims`) are created by the `Microsoft.AspNetCore.Identity.EntityFrameworkCore` migration. Two roles (`Admin`, `User`) are seeded in `OnModelCreating`; an admin and a regular test user are seeded by `SeedData.cs` on first run.
- **The schema is provider-agnostic.** It runs on SQLite locally (`api/BuckeyeMarketplace.db`) and Azure SQL in production via the same EF Core model.

User Guide

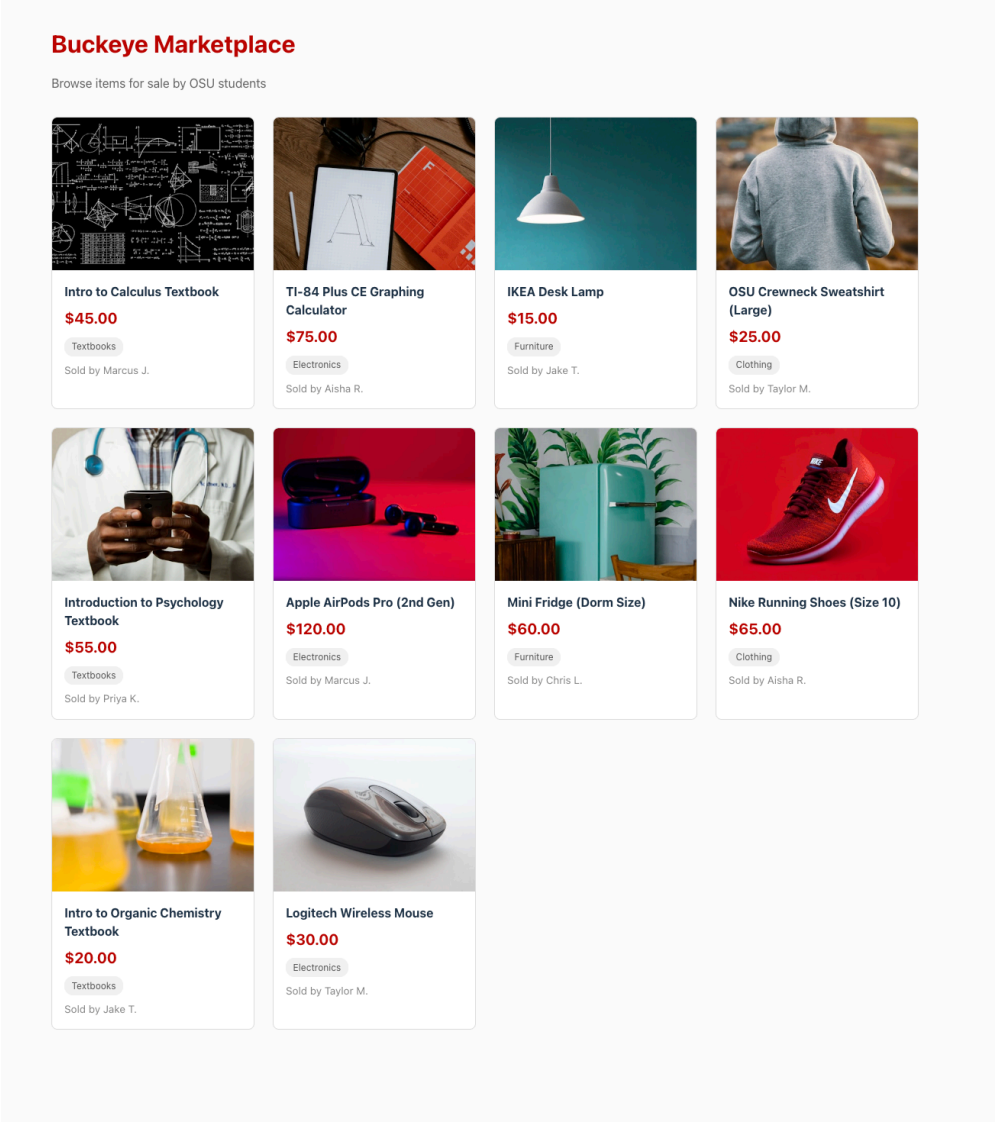
Source: `docs/user-guide.md`

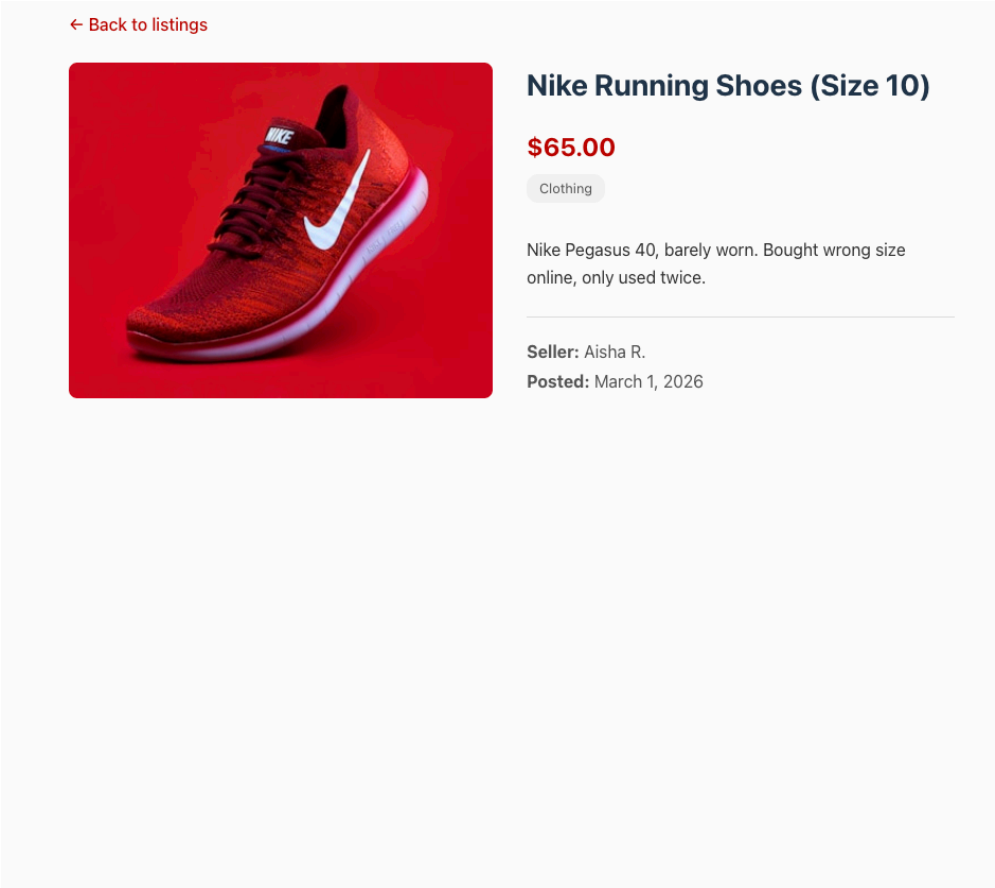
A safe place for OSU students to buy and sell. This guide walks through the five things a shopper does on the site.

> **Live site:** `https://buckeye-frontend-jama.azurewebsites.net` **Test login:** `user@buckeye.test` / `User1234!`

1. Browse products

Open the site. The catalog shows every listing as a card — title, price, category, seller. No login is needed to look around. Click any card to open the **product detail page** with the full description.





2. Create an account and log in

Click **Register** in the header. Enter an email and a password (minimum 8 characters with one uppercase letter and one digit). On success you're signed in automatically and your name appears in the header. Returning users click **Login** instead and use the email + password they registered with.

3. Add items to your cart

From either the catalog or a product detail page, click **Add to Cart**. The cart-count badge in the header increments. You can add the same item more than once; quantity is tracked per line.

Click the cart icon to open `/cart`. Each line item shows the unit price, a quantity control, and a remove button. The order summary on the right keeps the total live.

4. Place an order

From the cart click **Proceed to Checkout**. On `/checkout`:

1. Review the order summary on the right.
2. Fill in the shipping address.
3. Click **Place Order**.

You'll land on an **Order Confirmation** page that shows the confirmation number, total, and shipping address. Behind the scenes your cart is cleared and the line items are snapshotted into the order record.

5. View your order history

Click **Order History** in the header (or visit `/orders`). You'll see every order you've placed — confirmation number, date, total, status (`Pending` / `Processing` / `Shipped` / `Delivered` / `Cancelled`), and items. Order history is scoped to your account; you will only ever see your own orders.

To sign out, click **Logout** in the header. Your tokens are removed from the browser and you're returned to the catalog as a guest.

Admin Guide

Source: `docs/admin-guide.md`

Admin features cover **product management** and **order management**. Every admin action is protected on the server with `[Authorize(Roles = "Admin")]`.

> **Test admin login:** `admin@buckeye.test` / `Admin123!`

Sign in as an admin

Use the regular **Login** page with the admin credentials. You'll see an **Admin** link in the header that opens `/admin`.

Manage products

Visit `/admin/products`.

- **Add:** click **+ New Product**, fill in title, description, price, category, seller name, image URL, then **Save**. The product appears in the catalog immediately.
- **Edit:** click **Edit** on any row, change fields, **Save**. Past orders are unaffected — `OrderItem` rows snapshot product name, unit price, and line total at order time.
- **Delete:** click **Delete** and confirm. Past orders that referenced the product remain intact thanks to those snapshots.

Validation: price must be greater than zero; title and image URL are required.

Update an order's status

Visit `/admin/orders` to see every order in the system — confirmation number, customer email, date, total, current status.

Use the **Status** drop-down on a row to change the order: `Pending` → `Processing` → `Shipped` → `Delivered`. Any state can move to `Cancelled`. The change saves immediately (`PUT /api/orders/{id}/status`); the customer sees the new status the next time they refresh `/orders`.